

Advanced RAG

Complete Study Guide

Semantic Search • Hybrid Search • Metadata Filtering • Context Engineering

Data Science & AI Engineering Prep

1. What is Advanced RAG and Why Does It Exist?

Basic RAG (Retrieval Augmented Generation) gives the LLM a knowledge base to answer from. But basic RAG has serious problems in production — it retrieves the wrong documents, misses exact terms, returns chunks without context, or searches through irrelevant documents.

Advanced RAG is a collection of strategies that fix these problems. Think of it as upgrading from a simple Ctrl+F search to having a smart librarian who understands meaning, filters by category, and brings you full pages instead of random sentences.

Problem with Basic RAG	Advanced RAG Solution
Keyword matching misses meaning	Semantic Search — search by meaning, not words
Pure semantic misses exact terms/codes	Hybrid Search — semantic + keyword combined
Retrieved chunks lack surrounding context	Context Engineering — smarter chunking strategies
Searches all docs even irrelevant ones	Metadata Filtering — pre-filter by date, source, category

1.1 The Library Analogy

- Basic RAG = Ctrl+F on every book — finds exact words, misses meaning
- Semantic Search = a librarian who understands what you mean, not just what you said
- Hybrid Search = librarian + traditional index cards combined
- Metadata Filtering = 'only search books published after 2023 in the Finance section'
- Context Engineering = getting full paragraphs instead of random disconnected sentences

2. Semantic Search

2.1 What is Semantic Search?

Regular keyword search matches exact words. Semantic search matches meaning. If you search for 'automobile engine problems', semantic search also finds 'car motor issues' — because they mean the same thing, even though no words overlap.

Keyword Search	Semantic Search
Matches exact words only	Matches meaning and intent
Fast, no ML required	Requires embedding model
Great for exact terms, codes, IDs	Great for natural language questions
Fails on synonyms and paraphrases	Handles synonyms and paraphrases naturally
'car trouble' won't find 'vehicle issues'	'car trouble' WILL find 'vehicle issues'

2.2 How Semantic Search Works — Embeddings

Text is converted into vectors (lists of numbers) called embeddings. Similar meanings produce similar vectors that are close together in vector space. The search finds documents whose vectors are closest to the query vector — measured using cosine similarity.

```
# Conceptually, embeddings look like this:
"dog"      -> [0.20, 0.80, 0.11, 0.43, ...] # 384 numbers
"puppy"    -> [0.21, 0.79, 0.12, 0.44, ...] # very close = similar meaning
"automobile" -> [0.72, 0.15, 0.88, 0.31, ...] # far away = different meaning
"car"      -> [0.70, 0.16, 0.87, 0.30, ...] # close to automobile

# Cosine similarity: 1.0 = identical meaning, 0.0 = no relation, -1.0 = opposite
# similarity(dog, puppy)      = 0.97 (very similar)
# similarity(dog, automobile) = 0.12 (very different)
```

2.3 Code — Semantic Search with LlamaIndex

```
from llama_index.core import VectorStoreIndex, SimpleDirectoryReader
from llama_index.embeddings.huggingface import HuggingFaceEmbedding
from llama_index.core import Settings

# Step 1: Use a free local embedding model (no API key needed)
Settings.embed_model = HuggingFaceEmbedding(
    model_name="BAAI/bge-small-en-v1.5" # 33M params, fast and accurate
)

# Step 2: Load documents and build vector index
# Each chunk gets converted to an embedding vector and stored
documents = SimpleDirectoryReader("./docs").load_data()
index = VectorStoreIndex.from_documents(documents)
```

```
# Step 3: Query by meaning – finds by semantic similarity
query_engine = index.as_query_engine(
    similarity_top_k=3 # Retrieve top 3 most similar chunks
)
response = query_engine.query("What caused the revenue drop?")
print(response)

# similarity_top_k controls how many chunks are retrieved
# Higher k = more context but more noise
# Typical values: 3-5 for chat, 5-10 for research
```

2.4 Popular Embedding Models

Model	Size	Best For
BAAI/bge-small-en-v1.5	33M params, ~130MB	Fast local use, good accuracy
BAAI/bge-large-en-v1.5	335M params, ~1.3GB	Higher accuracy, slower
text-embedding-3-small	OpenAI API	Production, cloud-based
text-embedding-3-large	OpenAI API	Highest accuracy, cloud-based
sentence-transformers/all-MiniLM-L6-v2	22M params, ~90MB	Lightest, fastest local model

3. Hybrid Search

3.1 Why Hybrid Search?

Neither semantic nor keyword search is perfect alone. A medical system needs to find 'Metformin-500' (exact code — keyword wins) AND 'what helps with high blood sugar?' (meaning — semantic wins). Hybrid search combines both to get the best of both worlds.

	Semantic Search	Keyword Search (BM25)
Good at	Natural language, synonyms, meaning	Exact terms, codes, product IDs
Bad at	Exact codes, proper nouns, IDs	Understanding intent, paraphrases
Example win	'car trouble' finds 'vehicle issues'	'SKU-4821' finds exact product code
Example fail	Misses exact drug name 'Metformin'	Misses 'high blood sugar' for 'diabetes'
Algorithm	Cosine similarity on embeddings	BM25 term frequency scoring

3.2 How Hybrid Search Works — RRF Scoring

Both searches run in parallel. Each produces a ranked list. The lists are fused using Reciprocal Rank Fusion (RRF) — a formula that combines rankings with a configurable alpha weight.

```
# RRF Formula:
Final Score = (semantic_score x alpha) + (keyword_score x (1 - alpha))

# alpha examples:
alpha = 0.7 -> 70% semantic + 30% keyword (trust meaning more)
alpha = 0.3 -> 30% semantic + 70% keyword (trust exact terms more)
alpha = 0.5 -> 50% semantic + 50% keyword (balanced - good default)

# Choose alpha based on your use case:
# Natural language questions -> alpha closer to 0.7
# Exact term/code lookup      -> alpha closer to 0.3
# Mixed use case              -> alpha = 0.5
```

3.3 Code — Hybrid Search with LlamaIndex

```
from llama_index.core import VectorStoreIndex, SimpleDirectoryReader
from llama_index.core.retrievers import VectorIndexRetriever
from llama_index.retrievers.bm25 import BM25Retriever
from llama_index.core.retrievers import QueryFusionRetriever
from llama_index.embeddings.huggingface import HuggingFaceEmbedding
from llama_index.core import Settings

Settings.embed_model = HuggingFaceEmbedding(model_name='BAAI/bge-small-en-v1.5')
```

```
# Load and index documents
documents = SimpleDirectoryReader('./docs').load_data()
index = VectorStoreIndex.from_documents(documents)

# Retriever 1: Semantic (vector-based)
vector_retriever = VectorIndexRetriever(index=index, similarity_top_k=5)

# Retriever 2: Keyword - BM25 (same algorithm used by Elasticsearch & Google)
bm25_retriever = BM25Retriever.from_defaults(
    docstore=index.docstore,
    similarity_top_k=5
)

# Hybrid: fuse both retrievers with RRF
hybrid_retriever = QueryFusionRetriever(
    retrievers=[vector_retriever, bm25_retriever],
    similarity_top_k=3,          # Final top 3 after fusion
    num_queries=1,              # Do not generate extra queries
    mode="reciprocal_rerank",    # RRF fusion method
)

# Retrieve
nodes = hybrid_retriever.retrieve("revenue drop in Q3 India division")
for node in nodes:
    print(node.score, node.text[:100])
```

BM25 Note:

BM25 (Best Match 25) is the industry-standard keyword ranking algorithm. It ranks documents by how often the search terms appear, adjusted for document length. It powers Elasticsearch, Apache Solr, and is used internally by Google. Always prefer BM25 over simple TF-IDF for keyword search.

4. Metadata Filtering

4.1 What is Metadata Filtering?

Before semantic search even runs, filter documents by their metadata — extra information attached to each document like year, department, source file, author, or category. This is like a SQL WHERE clause applied before vector search.

Without filtering: searching 'What were the Q3 sales figures?' retrieves sales docs from 2019, 2020, 2021, 2022, 2023 all mixed together.

With metadata filter (year=2023, department=Sales): only 2023 Sales documents are searched, giving precise, relevant results.

4.2 Adding Metadata to Documents

```
from llama_index.core import Document

# Option 1: Add metadata manually to each document
doc = Document(
    text="Q3 2023 revenue for India division was $4.2M, up 12% YoY.",
    metadata={
        "year": 2023,
        "quarter": "Q3",
        "department": "Finance",
        "region": "India",
        "source": "annual_report_2023.pdf"
    }
)

# Option 2: Auto-attach filename as metadata via SimpleDirectoryReader
from llama_index.core import SimpleDirectoryReader
documents = SimpleDirectoryReader(
    "./docs",
    file_metadata=lambda filepath: {
        "source": filepath,
        "year": 2023
    }
).load_data()
```

4.3 Code — Filtering at Query Time

```
from llama_index.core import VectorStoreIndex
from llama_index.core.vector_stores import MetadataFilter, MetadataFilters
from llama_index.core.vector_stores.types import FilterOperator, FilterCondition

# Build index (documents already have metadata attached)
index = VectorStoreIndex.from_documents(documents)

# Define filters
filters = MetadataFilters(
```

```
filters=[
    MetadataFilter(key="year",      value=2023),
    MetadataFilter(key="department", value="Finance"),
],
condition=FilterCondition.AND # Both conditions must match
)

# Query engine with filters - ONLY searches matching documents
query_engine = index.as_query_engine(
    similarity_top_k=3,
    filters=filters
)

response = query_engine.query("What was the revenue in Q3?")
print(response)

# Filter with OR condition - match either Finance OR Sales
filters_or = MetadataFilters(
    filters=[
        MetadataFilter(key="department", value="Finance"),
        MetadataFilter(key="department", value="Sales"),
    ],
    condition=FilterCondition.OR
)
```

4.4 Why Metadata Filtering is Fast

Semantic search requires computing cosine similarity between the query vector and every stored vector — expensive for large datasets. Metadata filtering is just a dictionary lookup — checking if a field equals a value. No math, no vectors. This pre-filtering dramatically reduces the number of documents the semantic search must compare against.

Without Metadata Filter	With Metadata Filter
Semantic search on ALL 100,000 docs	Filter to 500 matching docs, then semantic search on 500
Slow, retrieves irrelevant results	Fast, results are scoped to relevant documents
Like searching entire library	Like searching only the Finance shelf, 2023 section

5. Context Engineering

5.1 The Problem with Naive Chunking

Even with great retrieval, the quality of chunks matters enormously. Naive chunking (splitting every N tokens) often creates chunks that are meaningless without surrounding context.

```
# Original paragraph:
"The board approved the merger. It was valued at $2.3B.
The deal closed in Q4 2023."

# Naive chunks (split every ~15 tokens):
Chunk 1: "The board approved the merger."
Chunk 2: "It was valued at $2.3B."    <- What is "It"? Useless without context!
Chunk 3: "The deal closed in Q4 2023."

# Problem: If Chunk 2 is retrieved, the LLM sees '$2.3B' but has no idea WHAT
# was valued at $2.3B. The answer will be wrong or hallucinated.
```

5.2 Strategy 1 — Sentence Window Retrieval

Store each sentence individually for precise retrieval. But when a sentence is found, automatically return the surrounding window of sentences (e.g., 3 before and 3 after) so the LLM gets full context.

```
from llama_index.core.node_parser import SentenceWindowNodeParser
from llama_index.core.postprocessor import MetadataReplacementPostProcessor
from llama_index.core import VectorStoreIndex

# Step 1: Parse documents into sentence-level nodes
# Each node stores the sentence AND a window of neighbors in metadata
parser = SentenceWindowNodeParser.from_defaults(
    window_size=3    # Include 3 sentences before and after the matched sentence
)
nodes = parser.get_nodes_from_documents(documents)

# Step 2: Build index on sentence-level nodes (precise retrieval)
index = VectorStoreIndex(nodes)

# Step 3: Query engine - retrieves sentence, then replaces with window
query_engine = index.as_query_engine(
    similarity_top_k=3,
    node_postprocessors=[
        # After retrieval, swap the sentence for its surrounding window
        MetadataReplacementPostProcessor(target_metadata_key="window")
    ]
)

response = query_engine.query("What was the merger value?")
print(response)
```

```
# How it works:
# Search: finds Chunk 2 'It was valued at $2.3B' (precise match)
# Replace: swaps it with window = [Chunk1 + Chunk2 + Chunk3] (full context)
# LLM receives: full paragraph -> correct answer
```

5.3 Strategy 2 — Auto-Merging (Hierarchical) Retrieval

Store documents at multiple chunk sizes simultaneously — small chunks for precise retrieval, large chunks for full context. When enough small chunks from the same parent are retrieved, automatically merge them into the large parent chunk before sending to the LLM.

```
# Hierarchical chunk structure:
Large chunk (512 tokens): [Full paragraph about the merger deal]
+-- Small chunk 1 (128 tokens): 'The board approved the merger.'
+-- Small chunk 2 (128 tokens): 'It was valued at $2.3B.'
+-- Small chunk 3 (128 tokens): 'The deal closed in Q4 2023.'

# Retrieve by small chunks (precise) -> enough found? -> merge into large (full context)
```

```
from llama_index.core.node_parser import HierarchicalNodeParser, get_leaf_nodes
from llama_index.core.retrievers import AutoMergingRetriever
from llama_index.core import VectorStoreIndex, StorageContext

# Step 1: Create nodes at multiple levels: 512, 256, 128 tokens
parser = HierarchicalNodeParser.from_defaults(
    chunk_sizes=[512, 256, 128]  # Largest to smallest
)
nodes = parser.get_nodes_from_documents(documents)

# Step 2: Index only LEAF nodes (smallest chunks for precise retrieval)
leaf_nodes = get_leaf_nodes(nodes)
storage_context = StorageContext.from_defaults()
storage_context.docstore.add_documents(nodes)  # Store ALL levels for merging

index = VectorStoreIndex(leaf_nodes, storage_context=storage_context)

# Step 3: Auto-merging retriever
base_retriever = index.as_retriever(similarity_top_k=6)
retriever = AutoMergingRetriever(
    base_retriever,
    storage_context,
    verbose=True
)

# Step 4: Retrieve - small chunks found -> auto-merged -> large context returned
nodes = retriever.retrieve("What was the merger value?")
for node in nodes:
    print(node.text[:200])

# Why index only leaf nodes?
# Small chunks = precise semantic match (find exact relevant sentence)
```

```
# Large chunks = full context for LLM to understand and answer
# Auto-merger bridges the gap automatically
```

5.4 Sentence Window vs Auto-Merging — When to Use Which

Sentence Window	Auto-Merging
Fixed window around matched sentence	Dynamic merging based on retrieval patterns
Simpler to set up, fewer moving parts	More sophisticated, better for long documents
Good for medium-length documents	Better for long reports, books, legal docs
Window size is fixed (e.g., always 3 before/after)	Merge threshold is dynamic based on how many siblings retrieved
Use when document paragraphs are self-contained	Use when documents have deep hierarchical structure

6. Full Production RAG Pipeline

In production, you combine all four advanced strategies. Here is a complete pipeline that uses Sentence Window chunking, Metadata Filtering, and a hybrid-style query engine together.

```
from llama_index.core import VectorStoreIndex, SimpleDirectoryReader, Settings
from llama_index.embeddings.huggingface import HuggingFaceEmbedding
from llama_index.core.node_parser import SentenceWindowNodeParser
from llama_index.core.vector_stores import MetadataFilter, MetadataFilters
from llama_index.core.postprocessor import MetadataReplacementPostProcessor
from llama_index.llms.groq import Groq

# — 1. Configure LLM and Embedding Model —————
Settings.llm = Groq(model='llama3-8b-8192', api_key='your_groq_key')
Settings.embed_model = HuggingFaceEmbedding(model_name='BAAI/bge-small-en-v1.5')

# — 2. Load Documents with Metadata —————
documents = SimpleDirectoryReader(
    './docs',
    file_metadata=lambda path: {
        'source': path,
        'year': 2023,
        'department': 'Finance'
    }
).load_data()

# — 3. Context Engineering - Sentence Window Chunking —————
parser = SentenceWindowNodeParser.from_defaults(window_size=3)
nodes = parser.get_nodes_from_documents(documents)

# — 4. Build Vector Index —————
index = VectorStoreIndex(nodes)

# — 5. Metadata Filtering - only search relevant docs —————
filters = MetadataFilters(filters=[
    MetadataFilter(key='year', value=2023),
    MetadataFilter(key='department', value='Finance'),
])

# — 6. Assemble Full Query Engine —————
query_engine = index.as_query_engine(
    similarity_top_k=3,
    filters=filters,
    node_postprocessors=[
        MetadataReplacementPostProcessor(target_metadata_key='window')
    ]
)

# — 7. Query —————
response = query_engine.query('What was the Q3 revenue in India?')
print(response)

# Flow:
# Query -> Metadata Filter (fast dict lookup, narrows scope)
```

```
# -> Semantic Search (on filtered docs only)
# -> Window Expansion (swap sentence for surrounding context)
# -> LLM (generates answer from full context)
```

7. Advanced RAG Strategies — Comparison

Strategy	What It Solves	Key Parameter
Semantic Search	Finds meaning, not just keywords	similarity_top_k (3-5 typical)
Keyword Search (BM25)	Finds exact terms, codes, product IDs	similarity_top_k
Hybrid Search	Best of both — meaning + exact terms	alpha (0.5 default, 0.7 for semantic-heavy)
Metadata Filtering	Scopes search to relevant docs only	MetadataFilter key/value pairs
Sentence Window	Preserves context around matched sentence	window_size (3 is good default)
Auto-Merging	Dynamic context from hierarchical chunks	chunk_sizes ([512, 256, 128])

7.1 When to Use What

- Small doc set (<1000 docs), natural language queries -> Semantic Search alone is enough
- Mixed queries (natural language + exact codes) -> Hybrid Search
- Large doc set with clear categories (by year, dept, region) -> Add Metadata Filtering
- Documents with dense, reference-heavy text -> Sentence Window Retrieval
- Long reports, books, legal documents -> Auto-Merging Retrieval
- Production system with all of the above -> Combine everything in one pipeline

8. Q&A Cards — Test Your Understanding

Q1 What is the difference between keyword search and semantic search?	Keyword search matches exact words — if you search 'car trouble' it only finds documents containing those exact words. Semantic search matches meaning — it converts text to embedding vectors and finds documents with similar meaning, so 'car trouble' also finds 'vehicle issues' or 'automobile engine problems'.
Q2 What is an embedding and why is it useful for search?	An embedding is a numerical representation (a vector of numbers) of text. Similar meanings produce similar vectors that are geometrically close in vector space. This allows search systems to find semantically related text even when no words overlap, by computing cosine similarity between the query vector and document vectors.
Q3 What is Hybrid Search and how does it combine semantic and keyword search?	Hybrid Search runs both semantic (vector) search and keyword (BM25) search in parallel, then fuses the ranked results using Reciprocal Rank Fusion (RRF). The formula is: $\text{Final Score} = (\text{semantic_score} \times \alpha) + (\text{keyword_score} \times (1 - \alpha))$. $\alpha=0.7$ trusts semantic more; $\alpha=0.3$ trusts keyword more; $\alpha=0.5$ is balanced.
Q4 Why is metadata filtering faster than semantic search?	Semantic search requires computing cosine similarity between the query vector and every stored document vector — expensive math on potentially millions of vectors. Metadata filtering is simply a dictionary/database lookup: check if <code>document.year == 2023</code> . No math, no vectors. It pre-narrows the search space before semantic search runs.
Q5 What problem does Sentence Window Retrieval solve?	Naive chunking creates isolated sentences that lose context. For example, 'It was valued at \$2.3B' is meaningless without surrounding sentences. Sentence Window stores each sentence for precise retrieval but attaches a window of neighboring sentences. When retrieved, the sentence is replaced with its full window — giving the LLM complete context.
Q6 In Auto-Merging Retrieval, why do we index only small chunks but store all chunk sizes?	Small chunks are indexed because they provide precise semantic matching — the search finds the exact relevant sentence. But small chunks lack context. All chunk sizes are stored so that when enough small chunks from the same parent are retrieved, the system auto-merges them into the large parent chunk, giving the LLM full context to answer correctly.
Q7 You are building a RAG system for 50,000 legal documents spanning 10 years and 5 practice areas. What strategies would you apply?	Apply all four strategies: (1) Metadata Filtering with year and practice_area fields to narrow scope before searching. (2) Hybrid Search because legal queries mix natural language with exact case numbers/statutes. (3) Auto-Merging because legal documents are long and hierarchically structured. (4) Sentence Window as fallback for shorter clauses. This gives scoped, precise, context-aware retrieval.

Q8
What is BM25 and why is it preferred for keyword search?

BM25 (Best Match 25) is the industry-standard keyword ranking algorithm. It ranks documents by how often search terms appear, adjusted for document length (so longer documents don't unfairly score higher). BM25 powers Elasticsearch, Apache Solr, and is used internally by Google. It consistently outperforms simple TF-IDF and is the default choice for any keyword retrieval component.

Q9
What is cosine similarity and how is it used in semantic search?

Cosine similarity measures the angle between two vectors — it returns 1.0 for identical direction (same meaning), 0.0 for perpendicular (unrelated), and -1.0 for opposite direction. In semantic search, the query text is embedded into a vector and cosine similarity is computed against all stored document vectors. The top-k documents with highest similarity scores are retrieved.

Q10
What does `similarity_top_k` control and how should you choose its value?

`similarity_top_k` controls how many chunks are retrieved and sent to the LLM as context. Higher k = more context (better recall, less likely to miss relevant info) but also more noise (irrelevant chunks may confuse the LLM). Typical values: 3-5 for conversational QA, 5-10 for research or summarization tasks. Too high can exceed context window limits and degrade answer quality.

9. Advanced RAG Cheat Sheet

Installation

```
pip install llama-index
pip install llama-index-embeddings-huggingface
pip install llama-index-retrievers-bm25
pip install llama-index-llms-groq
pip install sentence-transformers
```

Quick Reference Code

```
from llama_index.core import VectorStoreIndex, SimpleDirectoryReader, Settings
from llama_index.embeddings.huggingface import HuggingFaceEmbedding
from llama_index.core.node_parser import SentenceWindowNodeParser
from llama_index.core.vector_stores import MetadataFilter, MetadataFilters
from llama_index.core.postprocessor import MetadataReplacementPostProcessor
from llama_index.core.retrievers import VectorIndexRetriever, QueryFusionRetriever
from llama_index.retrievers.bm25 import BM25Retriever

# — Embedding model (free, local) —————
Settings.embed_model = HuggingFaceEmbedding(model_name='BAAI/bge-small-en-v1.5')

# — Semantic Search —————
index = VectorStoreIndex.from_documents(documents)
engine = index.as_query_engine(similarity_top_k=3)

# — Hybrid Search —————
vector_ret = VectorIndexRetriever(index=index, similarity_top_k=5)
bm25_ret = BM25Retriever.from_defaults(docstore=index.docstore,
similarity_top_k=5)
hybrid_ret = QueryFusionRetriever([vector_ret, bm25_ret],
similarity_top_k=3, num_queries=1, mode='reciprocal_rerank')

# — Metadata Filtering —————
filters = MetadataFilters(filters=[
    MetadataFilter(key='year', value=2023),
    MetadataFilter(key='dept', value='Finance'),
])
engine = index.as_query_engine(similarity_top_k=3, filters=filters)

# — Sentence Window —————
nodes =
SentenceWindowNodeParser.from_defaults(window_size=3).get_nodes_from_documents(docu
ments)
index = VectorStoreIndex(nodes)
engine = index.as_query_engine(similarity_top_k=3,

node_postprocessors=[MetadataReplacementPostProcessor(target_metadata_key='window')
])

# — Auto-Merging —————
from llama_index.core.node_parser import HierarchicalNodeParser, get_leaf_nodes
from llama_index.core.retrievers import AutoMergingRetriever
```

```
from llama_index.core import StorageContext
nodes =
HierarchicalNodeParser.from_defaults(chunk_sizes=[512,256,128]).get_nodes_from_documents(documents)
sc = StorageContext.from_defaults()
sc.docstore.add_documents(nodes)
index = VectorStoreIndex(get_leaf_nodes(nodes), storage_context=sc)
retriever = AutoMergingRetriever(index.as_retriever(similarity_top_k=6), sc)
```

Key Concepts Summary

Concept	One-Line Summary
Embedding	Text converted to a vector of numbers; similar meaning = close vectors
Cosine Similarity	Measures angle between vectors; 1.0=identical, 0.0=unrelated
Semantic Search	Find by meaning using embeddings and cosine similarity
BM25	Keyword ranking algorithm; industry standard for exact-term search
Hybrid Search	Fuse semantic + BM25 results using RRF with alpha weighting
Metadata Filtering	Pre-filter docs by year/dept/source before vector search
Sentence Window	Retrieve sentences precisely; expand to surrounding window for context
Auto-Merging	Small chunks for retrieval; auto-merge into parent for full context
similarity_top_k	Number of chunks retrieved; 3-5 for chat, 5-10 for research
alpha	Hybrid weight: 0.7=semantic-heavy, 0.3=keyword-heavy, 0.5=balanced

End of Advanced RAG Complete Guide